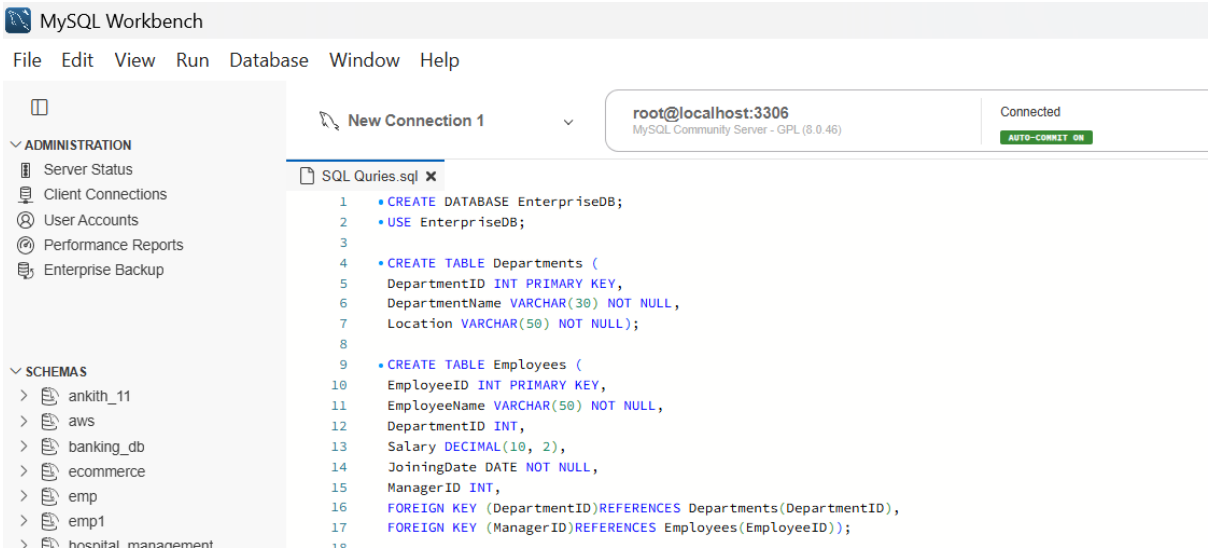
Ankith J
23BTLCN001
CSE-General
HCLTFP2262903
CLOUD DATA ENGINEER

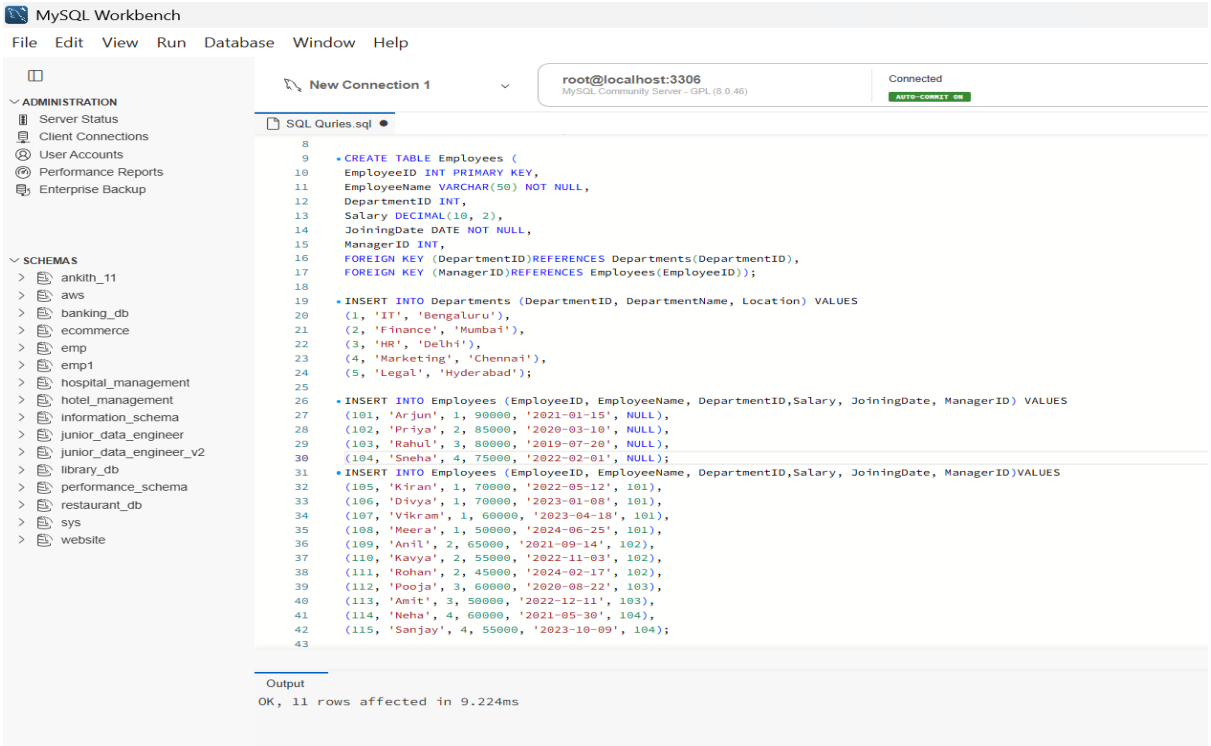
SECTION A: SQL

Question 1: Database Design and Queries

1.Create a database named EnterpriseDB.

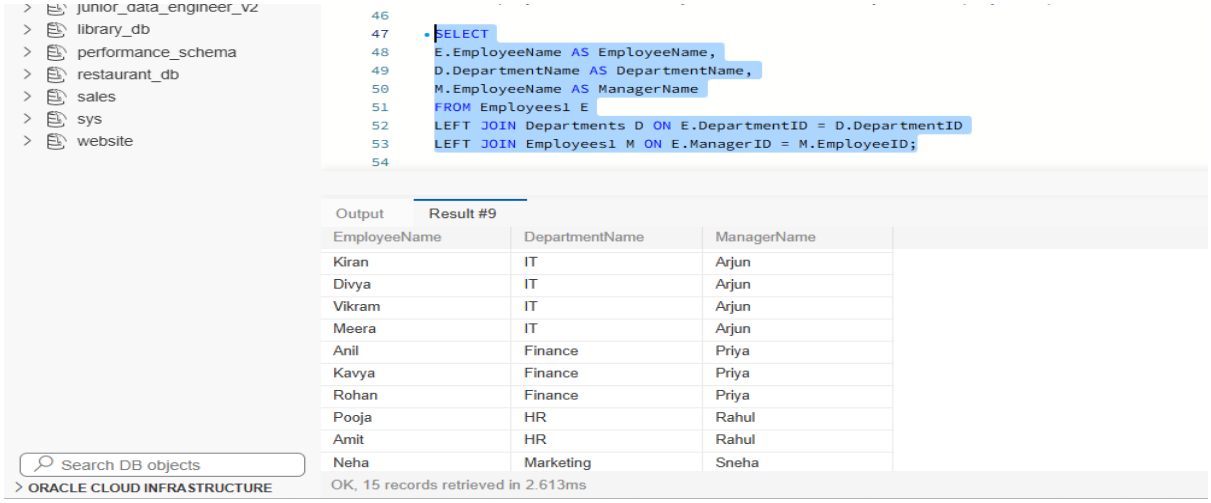


2.Inserting the Values

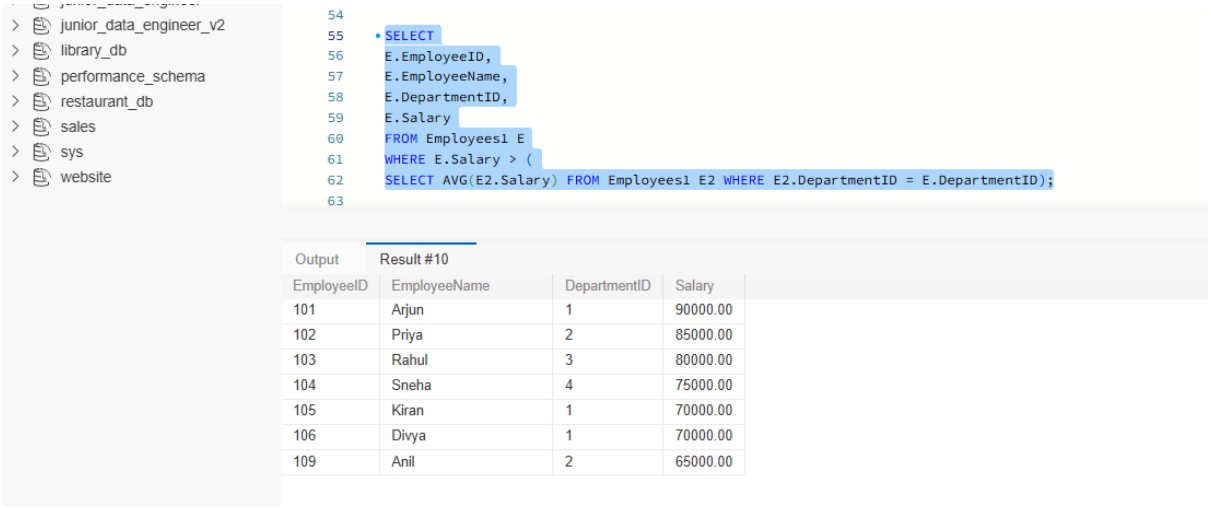


Tasks:

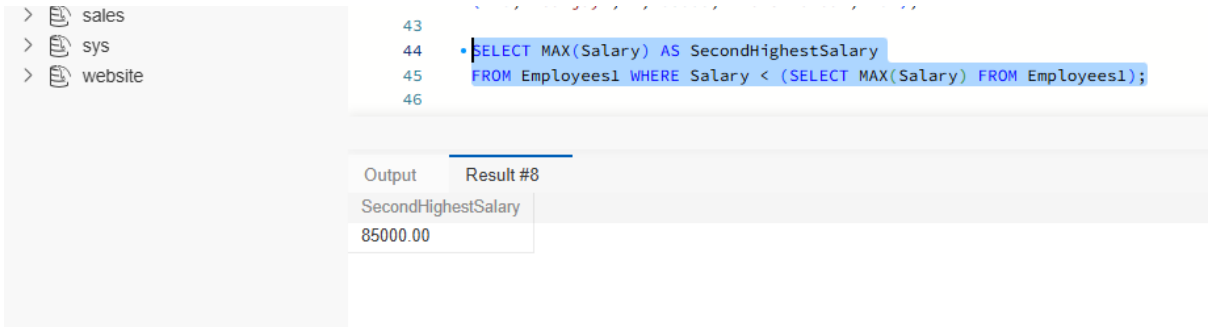
1. Display employee names, department names, and manager names using joins.



2. Find employees earning more than their department's average salary

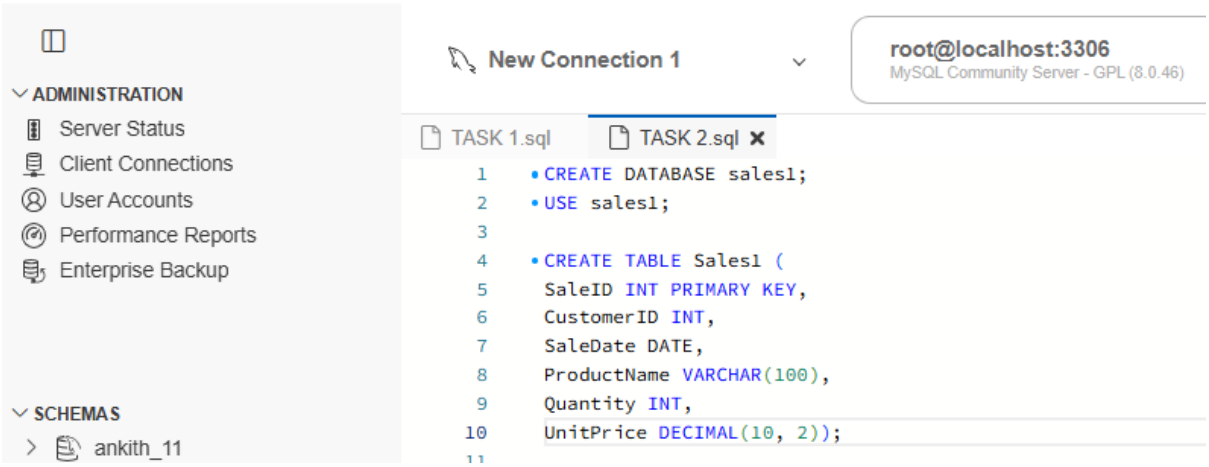


3. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST

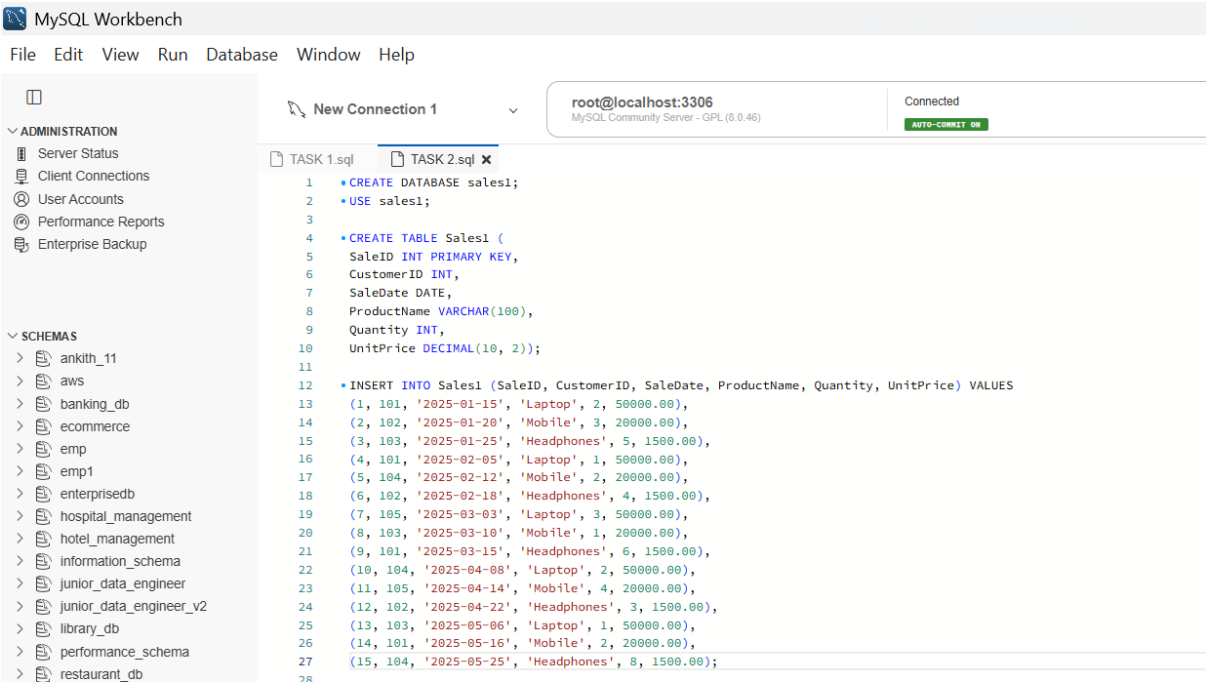


Question 2: Sales Analytics and Query Optimization

Create a Sales table

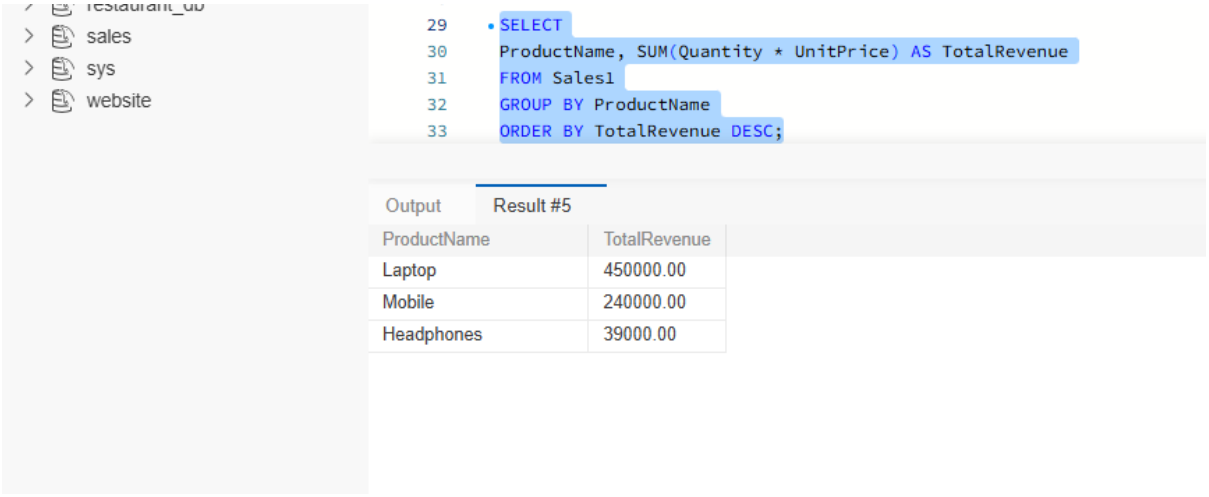


Insert at least 15 sample records covering multiple customers, products, and dates.

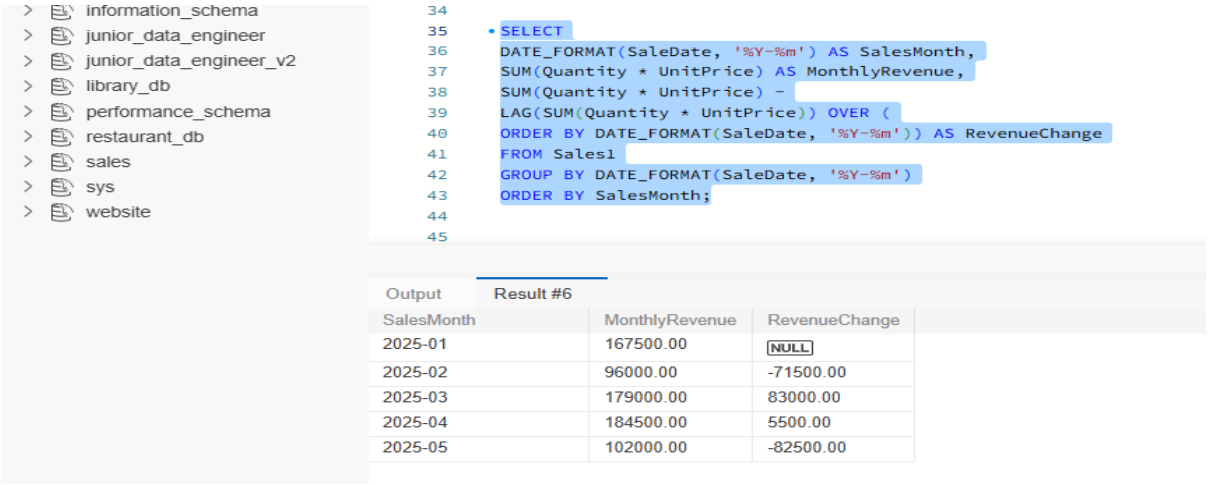


Tasks:

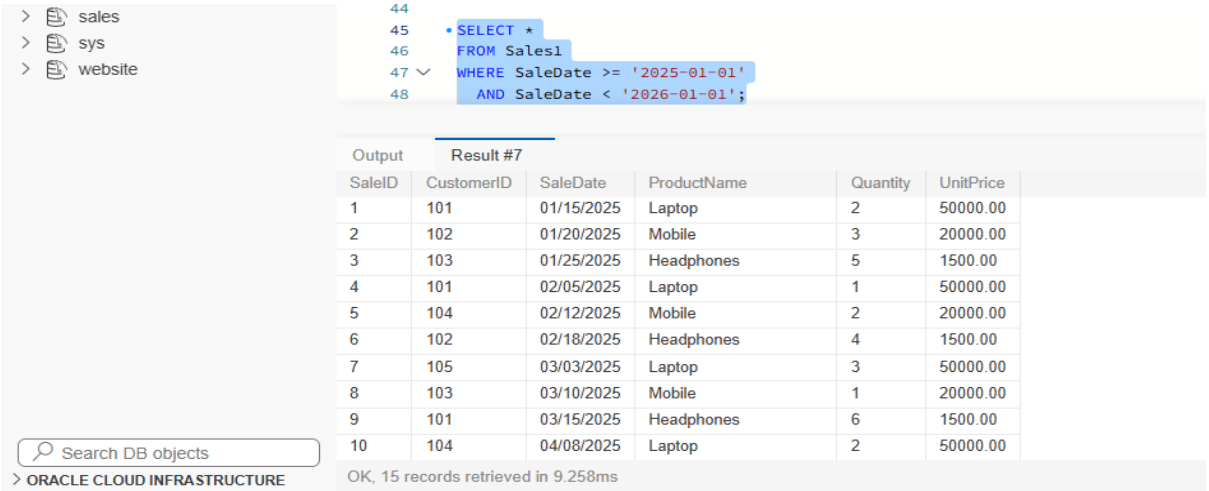
1.Calculate total revenue for each product.



2. Calculate monthly revenue and month-over-month revenue change using a window function.



3. Rewrite the following query to improve the potential for index usage and explain why your version may perform better.



Question 3: SQL Troubleshooting

1.Explain how an execution plan helps identify expensive operations in a query.

An execution plan shows how a database runs a query. It helps us find which operations are taking more time, such as scanning a whole table or joining large amounts of data. By identifying these issues, we can improve the query and make it run faster.

By analayzing the execution plan, we can identify operations with high cost and optimize the query using appropriate indexes and better query structure. This improves query performance and reduces execution time.

2.Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning.

We can create a combined index on DepartmentID and Salary. This helps the database find employees faster when searching by department and salary. It reduces the need to check every employee record and improves query performance.

```
CREATE INDEX idx_department_salary

ON Employees (DepartmentID, Salary);
```

3. Write a query to find duplicate customer email addresses.

This query groups customers with the same email address and counts how many times each address appears. The having condition displays only those email addresses that occur more than once.

```
SELECT Email, COUNT(*) AS DuplicateCount

FROM Customers

GROUP BY Email

HAVING COUNT(*) > 1;
```

SECTION B: LINUX

Question 4: Log Analysis

1.Find all lines containing ERROR

```
grep "ERROR" /var/log/application.log
```

2.Display the 20 most recent error entries in the file.

```
grep "ERROR" /var/log/application.log | tail -n 20
```

3. Count the total number of error entries.

```
grep -c "ERROR" /var/log/application.log
```

4. Extract error entries for a specified date into a separate file.

```
grep "^2026-10-28 .*ERROR" /var/log/application.log > /var/log/errors_2026-10-28.log
```

5. Explain how you would calculate the number of errors occurring in each hour.

```
grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c
```

Question 5: Permissions and System Troubleshooting

1.Create a group named finance and a report file, assuming you have the required privileges.

```
sudo groupadd finance
```

```
touch report.txt
```

2. Set permissions so the owner can read and write, the finance group can read, and other users have no access.

```
sudo chown "$USER":finance report.txt
```

```
chmod 640 report.txt
```

3. Identify the processes consuming the most CPU.

```
Top
```

4. Display memory usage and filesystem disk utilization.

```
free -h
```

```
df -h
```

5. Find files larger than 100 MB under a specified application directory

```
find /var/www/application -type f -size +100M
```

Question 6: Bash Scripting

SECTION C: PYSPARK DATA ENGINEERING

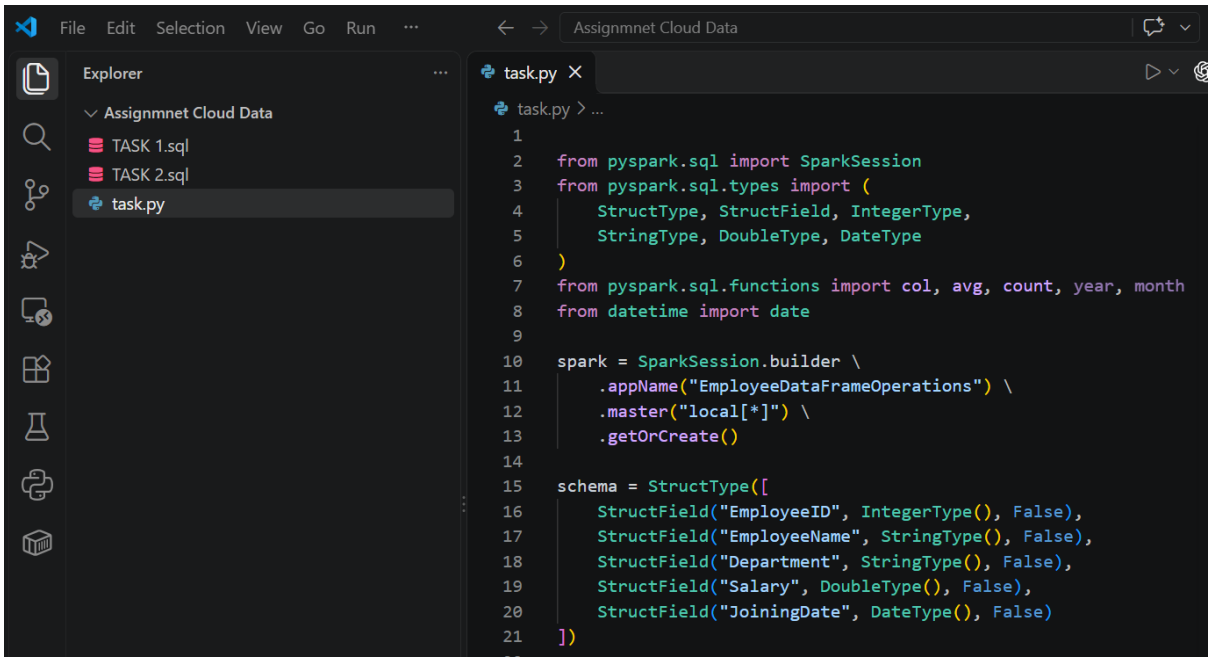
Accepts source and destination directories as command-line arguments.

```
#!/bin/bash
if [ "$#" -ne 2 ]; then
    echo "Error: Usage: $0 <source_directory> <destination_directory>"
    exit 1
fi
SOURCE="$1"
DESTINATION="$2"
if [ ! -d "$SOURCE" ]; then
    echo "Error: Source directory does not exist."
    exit 1
fi
mkdir -p "$DESTINATION" || {
    echo "Error: Could not create destination directory."
    exit 1}
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
ARCHIVE="$DESTINATION/backup_$TIMESTAMP.tar.gz"
if tar -czf "$ARCHIVE" -C "$(dirname "$SOURCE")" "$(basename "$SOURCE")"; then
    echo "Backup successful: $ARCHIVE"
else
    echo "Error: Backup failed."
    exit 1
fi
exit 0
```

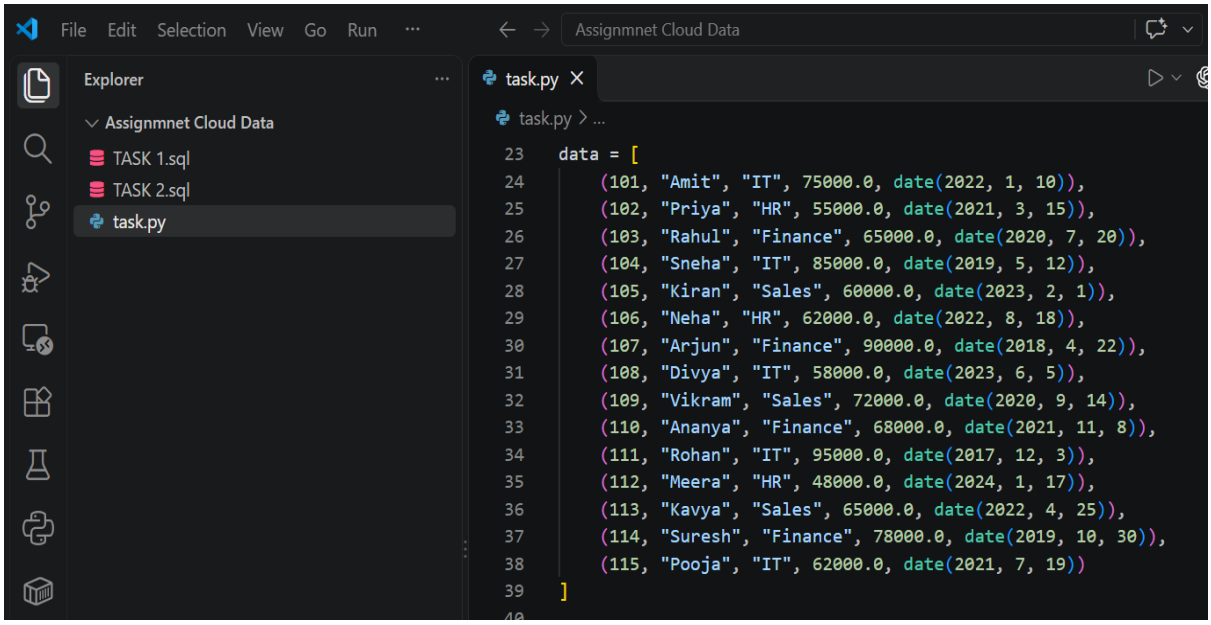
SECTION C: PYSPARK DATA ENGINEERING

Question 7: PySpark DataFrame Operations

Create a PySpark DataFrame

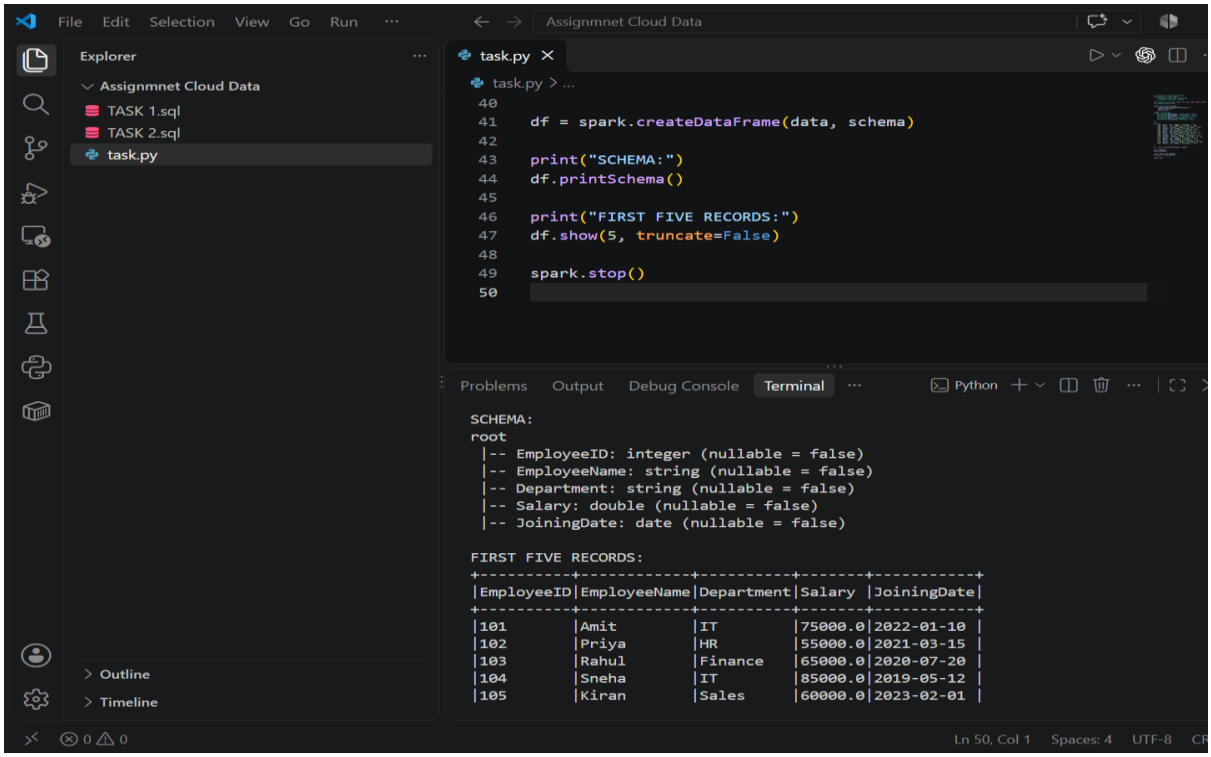


Inserting 15 Values

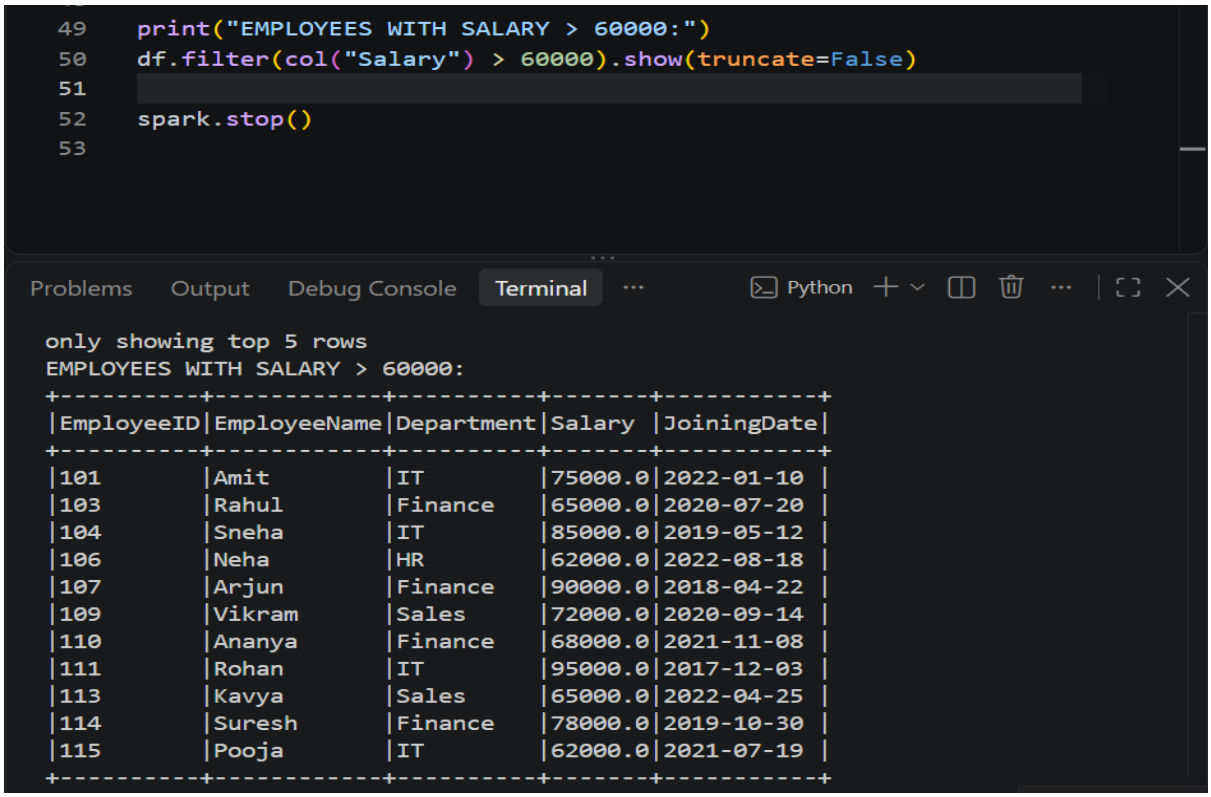


Tasks:

1.Display the schema and first five records



2.Filter employees whose monthly salary is greater than ₹60,000.



3.Add an Annual Salary column calculated from the monthly salary.

```
52    df_annual = df.withColumn(
53        "AnnualSalary", col("Salary") * 12 )
54
55    print("EMPLOYEES WITH ANNUAL SALARY:")
56    df_annual.show(truncate=False)
```

Problems Output Debug Console Terminal ··· Python

```
EMPLOYEES WITH ANNUAL SALARY:
+----------+------------+----------+-------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary |JoiningDate|AnnualSalary|
+----------+------------+----------+-------+-----------+------------+
|101       |Amit        |IT        |75000.0|2022-01-10 |900000.0    |
|102       |Priya       |HR        |55000.0|2021-03-15 |660000.0    |
|103       |Rahul       |Finance   |65000.0|2020-07-20 |780000.0    |
|104       |Sneha       |IT        |85000.0|2019-05-12 |1020000.0   |
|105       |Kiran       |Sales     |60000.0|2023-02-01 |720000.0    |
|106       |Neha        |HR        |62000.0|2022-08-18 |744000.0    |
|107       |Arjun       |Finance   |90000.0|2018-04-22 |1080000.0   |
|108       |Divya       |IT        |58000.0|2023-06-05 |696000.0    |
|109       |Vikram      |Sales     |72000.0|2020-09-14 |864000.0    |
```

4. Group employees by department and calculate the average salary and employee count.

```
59    print("DEPARTMENT-WISE SUMMARY:")
60    df.groupBy("Department").agg(
61        avg("Salary").alias("AverageSalary"),
62        count("*").alias("EmployeeCount")
63    ).orderBy("Department").show(truncate=False)
```

Problems Output Debug Console Terminal ··· Python

```
DEPARTMENT-WISE SUMMARY:
+----------+-----------------+-------------+
|Department|AverageSalary    |EmployeeCount|
+----------+-----------------+-------------+
|Finance   |75250.0          |4            |
|HR        |55000.0          |3            |
|IT        |75000.0          |5            |
|Sales     |65666.66666666667|3            |
+----------+-----------------+-------------+
```

5. Display the five highest-paid employees.

```
66    print("TOP FIVE HIGHEST-PAID EMPLOYEES:")
67    df.orderBy(col("Salary").desc()).limit(5).show(
68        truncate=False
69    )
70    spark.stop()
```

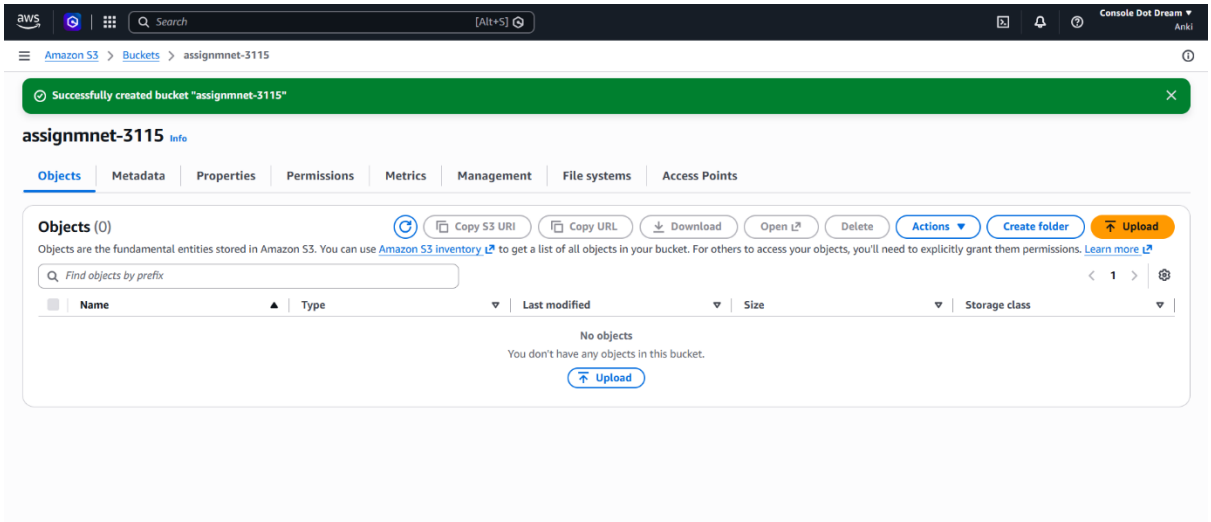
Problems Output Debug Console Terminal ··· Python

```
TOP FIVE HIGHEST-PAID EMPLOYEES:
+----------+------------+----------+-------+-----------+
|EmployeeID|EmployeeName|Department|Salary |JoiningDate|
+----------+------------+----------+-------+-----------+
|111       |Rohan       |IT        |95000.0|2017-12-03 |
|107       |Arjun       |Finance   |90000.0|2018-04-22 |
|104       |Sneha       |IT        |85000.0|2019-05-12 |
|114       |Suresh      |Finance   |78000.0|2019-10-30 |
|101       |Amit        |IT        |75000.0|2022-01-10 |
+----------+------------+----------+-------+-----------+
```

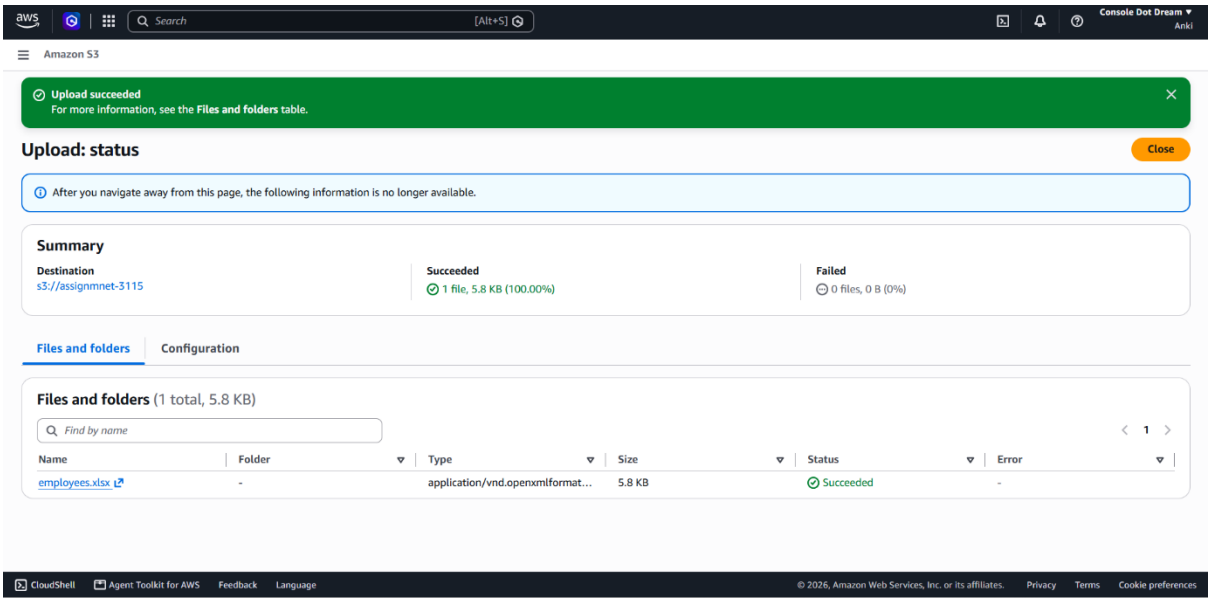
SECTION D: AWS CLOUD SERVICES

Question 10: S3 and IAM Security

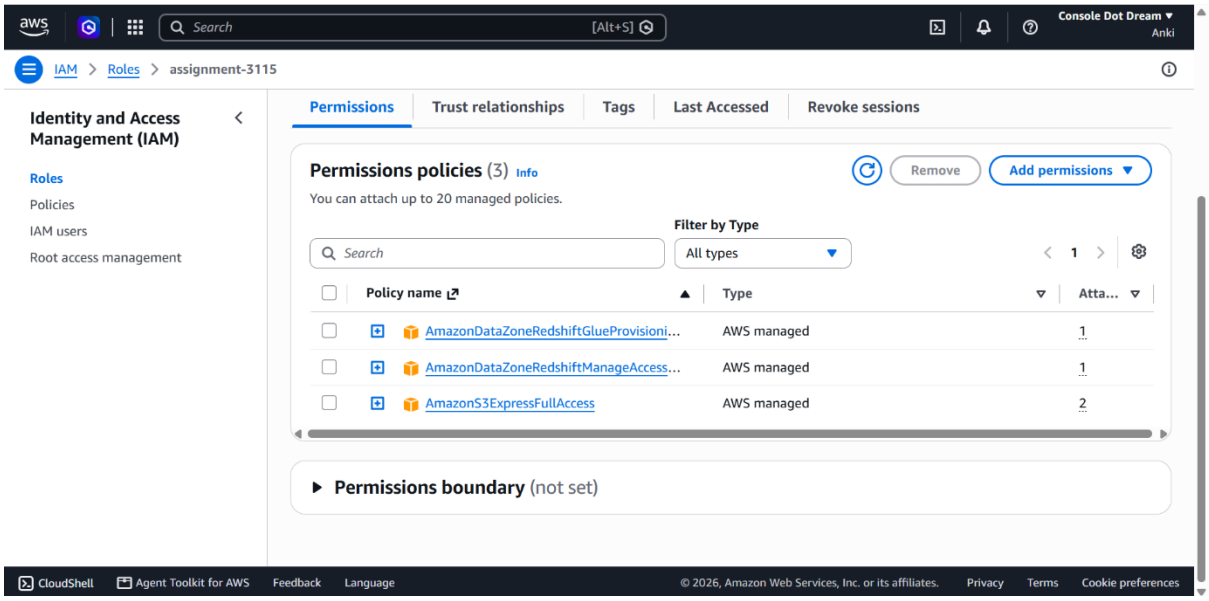
1.Create an S3 bucket with Block Public Access enabled and upload a sample file.



2.File uploaded succesfully



3.Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



3. Verify Unauthorized Access Is Denied

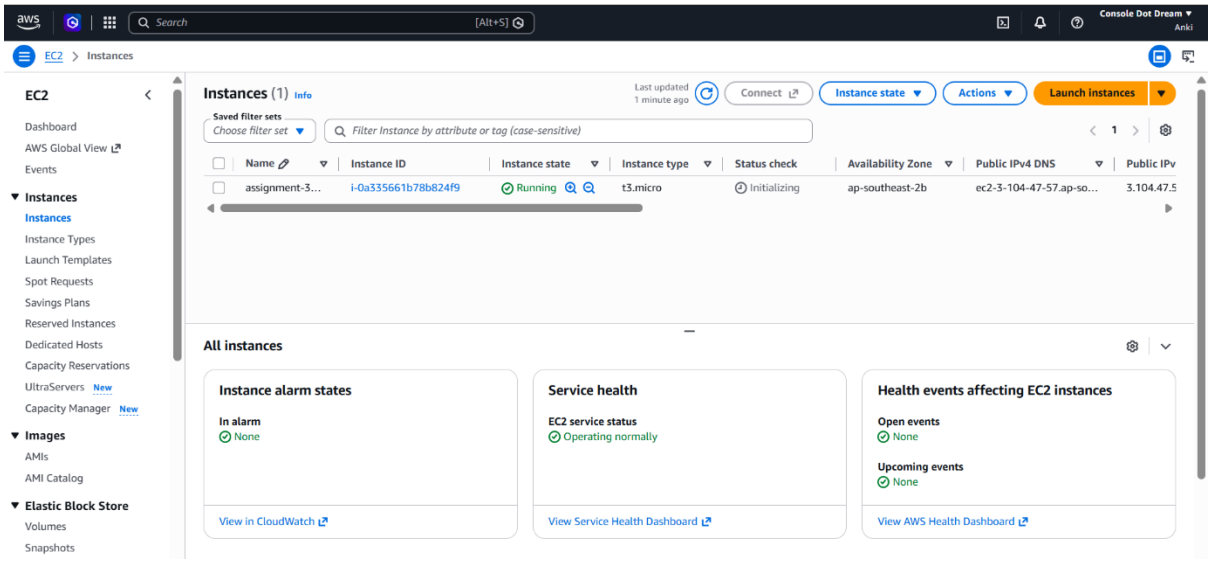
Log in as an unauthorized IAM user who does not have permission to access the bucket. Try to open or download the file. Verify that AWS returns an Access Denied (403) error, confirming that the user cannot read the file.

4. Encryption at Rest

Encryption at rest protects stored data by converting it into unreadable ciphertext using encryption keys. Amazon S3 supports server-side encryption, such as SSE-S3 and SSE-KMS. The data is decrypted when accessed by an authorized request, helping protect confidential employee reports from unauthorized access.

Question 11: EC2 Troubleshooting and CloudWatch

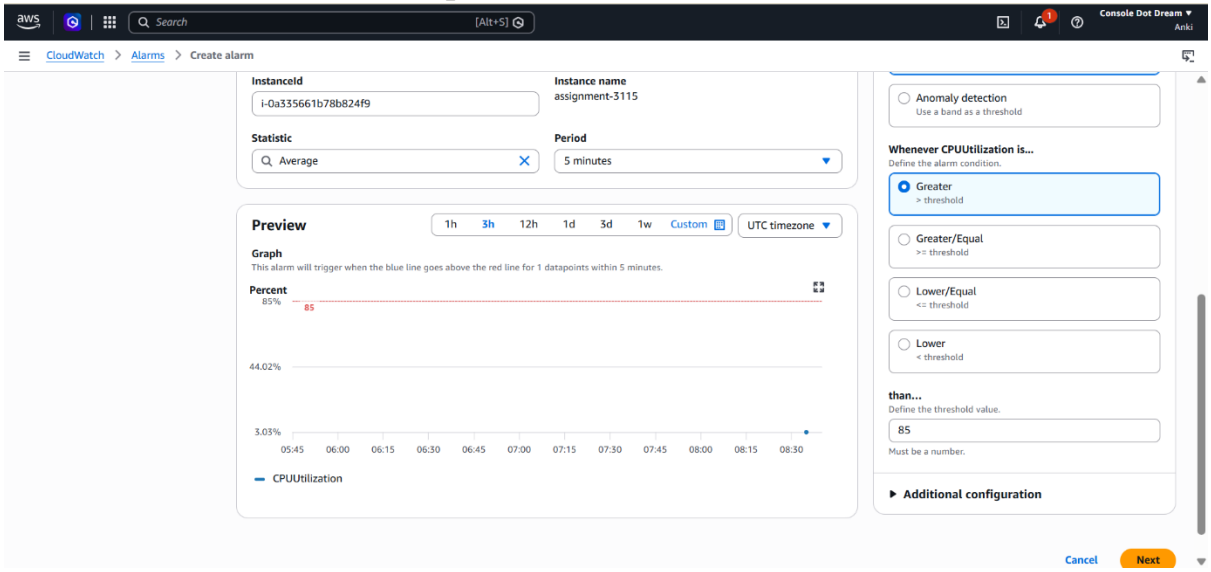
1.Check the EC2 instance state and status checks.



2. Explain how security groups and route tables can affect application connectivity.

- Security Groups: Incorrect inbound or outbound rules can block application traffic. Allow the required ports, such as HTTP (80) and HTTPS (443).
- Route Tables: Incorrect routes or a missing Internet Gateway route can prevent users from connecting to the application.

3. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.



4.Identify one method for investigating the root cause of high CPU utilization.

Use CloudWatch Metrics to identify when CPU utilization increases. Connect to the EC2 instance using SSH or AWS Systems Manager and run the top command to identify processes consuming excessive CPU.

Question 12: Event-Driven Processing

1.Identify a suitable service for executing the processing code.

Use AWS Lambda to execute code that automatically validates and processes CSV files uploaded to Amazon S3.

2.Explain how an S3 upload can trigger processing.

Configure an S3 Event Notification for the object created event. Whenever a CSV file is uploaded to the input bucket, Amazon S3 automatically triggers the Lambda function.

3.Explain the IAM permissions needed to read the input file and write the results.

Create an IAM execution role for Lambda with the following permissions:

- CloudWatch Logs permissions – to record execution logs.

Grant permissions only to the required buckets and objects.

4.Explain how errors and processing logs can be captured.

Use Amazon CloudWatch Logs to record processing details, errors, and execution failures. Configure CloudWatch alarms to notify administrators when failures occur. Lambda retries or a dead-letter queue (DLQ) can help handle failed processing.

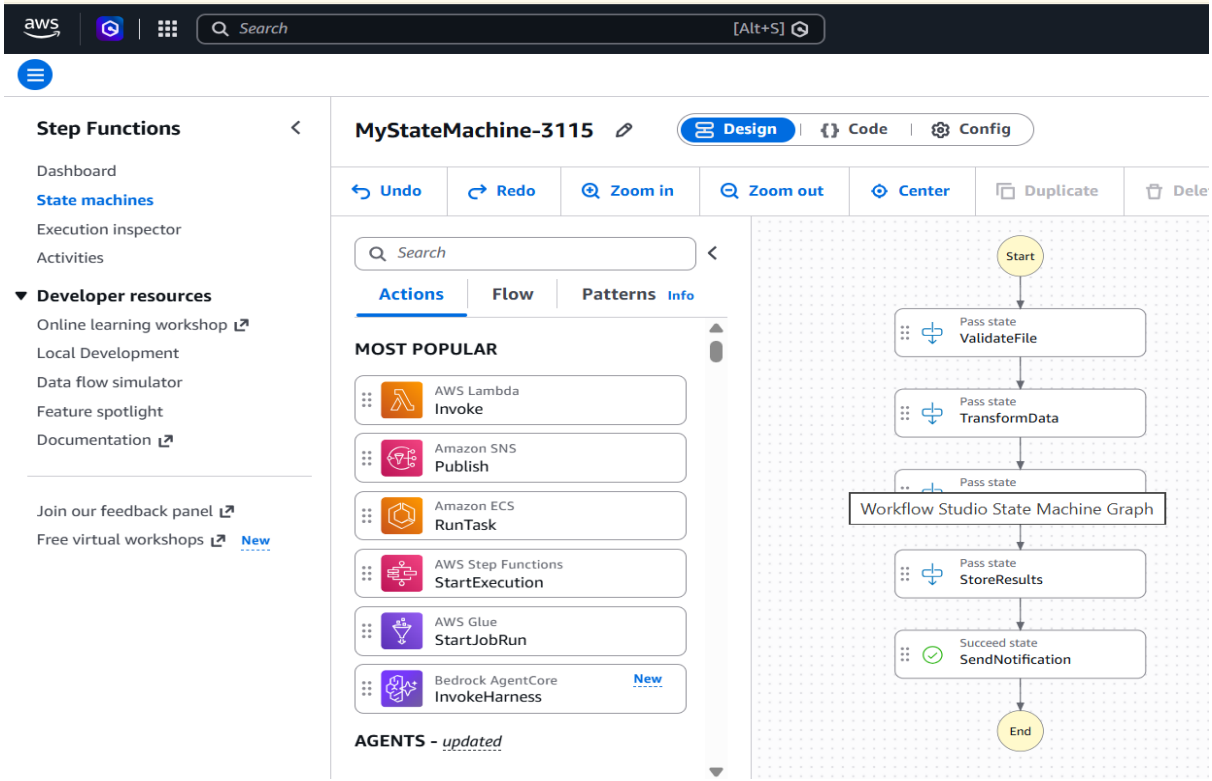
5.Explain how to prevent duplicate processing or recursive triggering.

- Use separate input and output buckets, or configure the trigger only for an input prefix.
- Store processed file identifiers in Amazon DynamoDB to prevent duplicate processing.
- Ensure that output files do not trigger the same Lambda function again.

SECTION E: AWS STEP FUNCTIONS

Question 13: State Machine Design

1.Identify suitable states for validation, processing, storage, and notification.



2.Explain how a Choice state can route invalid files to a failure path and valid files to processing.

A Choice state checks the validation result. If the file is valid (isValid = true), the workflow proceeds to processing. If the file is invalid (isValid = fail), it routes to a Fail state and stops the workflow.

3.Explain how Retry and Catch can handle transient failures and permanent errors.

- Retry: Automatically retries a task when temporary errors occur, such as network issues or service throttling. The retry policy specifies the interval, maximum attempts, and backoff rate.
- Catch: Handles errors that persist after retries or cannot be retried. It redirects the workflow to an error-handling state to log the error or notify an administrator.

4.Explain how the state machine can invoke Lambda functions or other supported AWS services

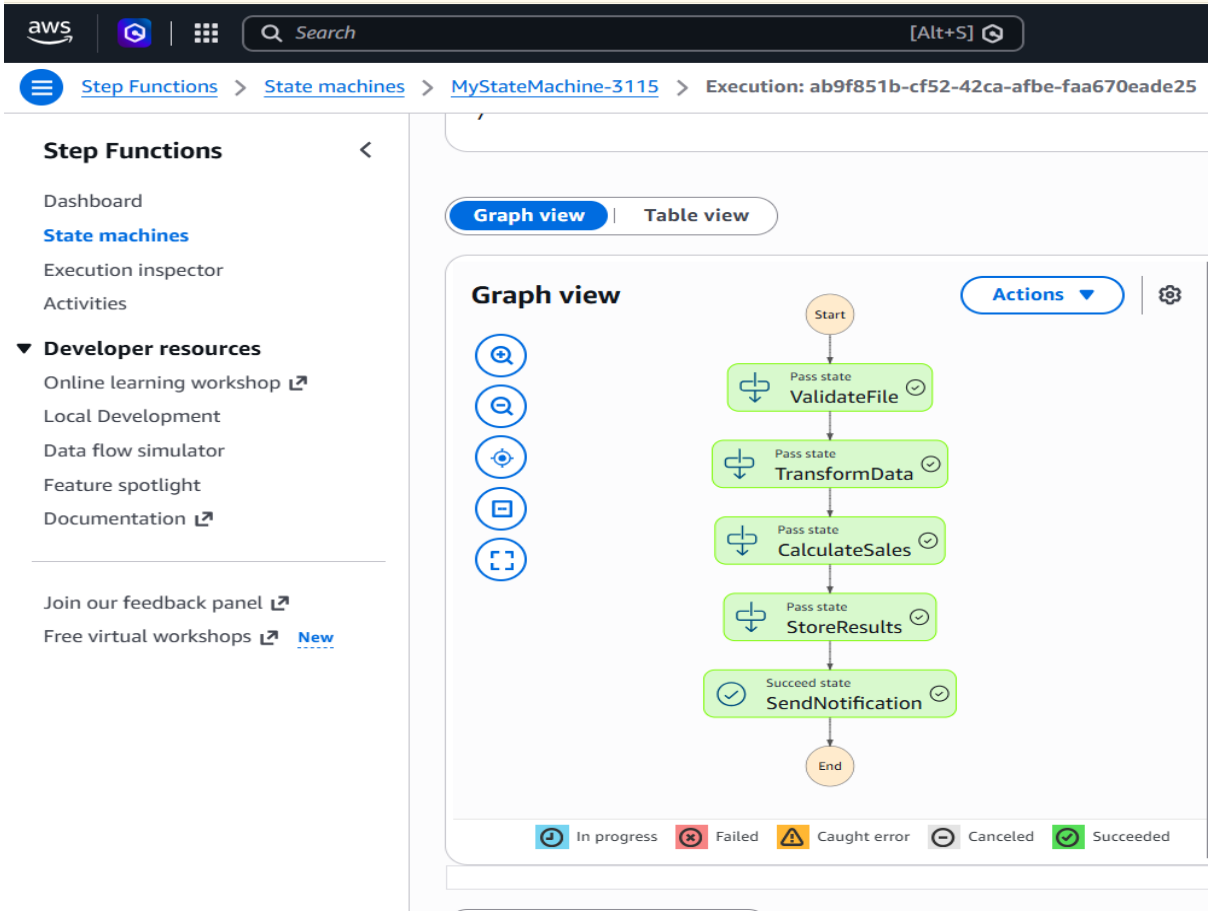
A Task state invokes an AWS Lambda function using its ARN or integrates with supported AWS services such as Amazon SNS and Amazon S3. This allows each processing stage to run automatically in sequence.

5.Explain how execution history helps troubleshoot a failed workflow.

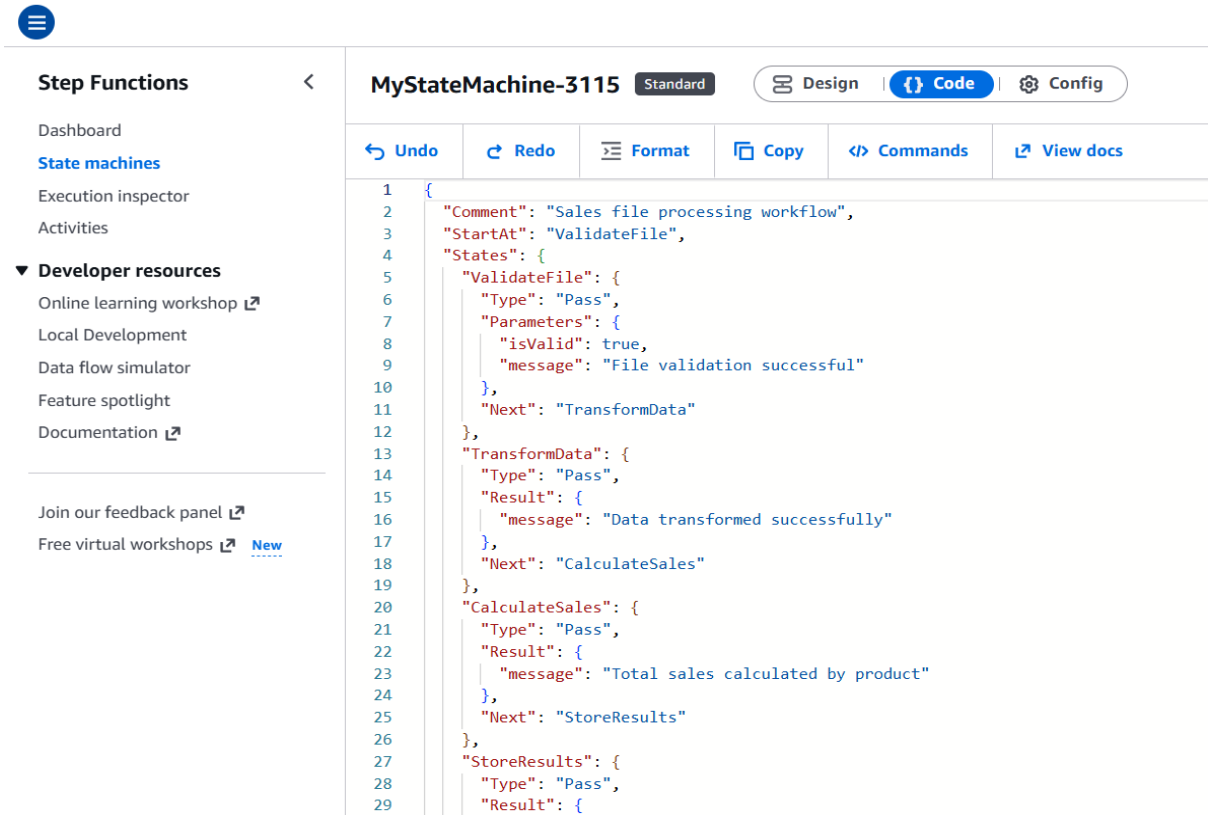
Execution history records each state's status, input, output, and error details. In the Step Functions console, administrators can identify the failed state, investigate the cause, and troubleshoot the workflow.

Question 14: Step Functions Implementation

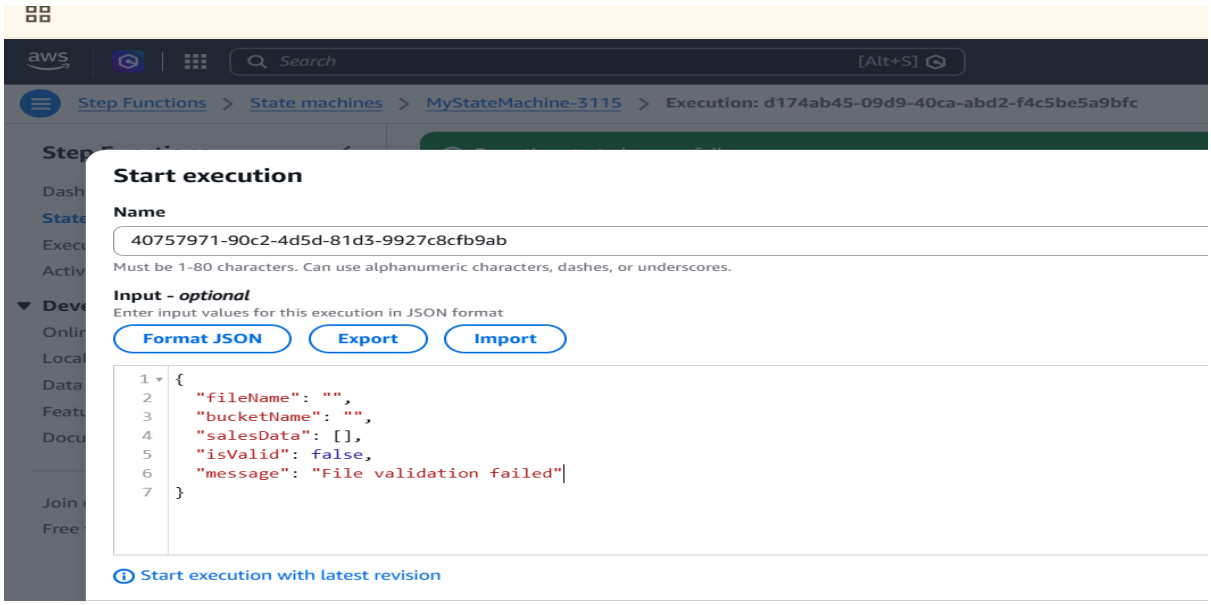
1.Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.



2. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.



3. Configure a Catch transition for the loading task that routes to an error-handling state.



4. Explain how data from one task is passed to the next task.

In AWS Step Functions, data is passed between states using JSON input and output.

- InputPath: Selects which part of the input is passed to a state.
- Parameters: Creates or modifies the input sent to a state.
- Result: Defines the output produced by a state.
- ResultPath: Combines a state's result with its original input.
- OutputPath: Selects which part of the state output is passed to the next state.

In the sales file workflow, ValidateFile passes its result to TransformData, followed by CalculateSales, StoreResults, and SendNotification.

5. Explain how you would test successful and failed executions

Successful execution:

1. Open AWS Step Functions and select the state machine.
2. Click Start execution.
3. Provide valid JSON input containing the sales file details and sales data.
4. Start the execution and check the execution graph.
5. Verify that all states complete successfully and the execution status is Succeeded

SECTION F: MICROSOFT AZURE

Question 15: Azure Data Lake Storage and Security

1.Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2.

Create an Azure Storage Account in the Azure portal. Select the appropriate subscription, resource group, region, and performance settings. Enable the Hierarchical namespace option to use Azure Data Lake Storage Gen2.

2. Create a filesystem or container and upload a sample dataset.

Open the storage account and navigate to Containers. Create a container (filesystem), such as customer-data, and upload a sample CSV file named sales.csv.

3. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data.

Microsoft Entra ID authenticates users and applications. Azure role-based access control (RBAC) authorizes them to access Azure resources.

- Assign the Storage Blob Data Reader role to users who only need to read data.
- Assign the Storage Blob Data Contributor role to users who need to read, upload, and modify data.

4. Explain how you would verify that unauthorized users cannot access the dataset.

Sign in as a user without the required permissions and attempt to access the file. Verify that access is denied. Also check role assignments and ensure public access is disabled.

Question 16: Azure VM Troubleshooting

1.Verify the VM's provisioning and power states.

Open the Azure portal, select Virtual Machines, and check the VM's provisioning state and power state. Confirm that provisioning is Succeeded and the VM is Running.

2. Explain how NSG rules can prevent connectivity

Network Security Group (NSG) rules control inbound and outbound traffic. A rule that blocks the required port, such as TCP 22 for SSH or TCP 3389 for RDP, can prevent administrators from connecting to the VM.

3. Explain how subnet and routing configuration affect connectivity

Incorrect subnet configuration, route tables, or network routes can prevent traffic from reaching the VM. Check the VM's subnet, IP configuration, route table, and connectivity to the required destination.

4. Recommend a secure troubleshooting method

Use Azure Bastion to connect to the VM through the Azure portal without assigning the VM a public IP address. Check NSG rules, effective routes, and Azure Network Watcher connection troubleshooting tools. Grant access only to authorized administrators and avoid opening management ports to the entire internet.

Question 17: Azure Data Integration

1.Select an appropriate service to orchestrate data movement.

Use Azure Data Factory (ADF) to create and orchestrate a data pipeline for moving and processing customer transaction CSV files.

2. Explain how to copy the CSV file into Azure Data Lake Storage Gen2

Create a pipeline in Azure Data Factory and add a Copy Data activity. Configure the source dataset for the incoming CSV file and the sink dataset for Azure Data Lake Storage Gen2. Run the pipeline to copy the file.

3. Describe how to organize files by ingestion date

Store files in date-based folders to simplify tracking and analysis.

4. Explain how to handle malformed records and missing values

Use Azure Data Factory mapping data flows or another data-processing service to validate records, handle missing values according to business rules, and redirect malformed records to a separate error folder for investigation.

5. Explain how to monitor pipeline execution and validate data completeness

Use Azure Data Factory's Monitor section to check pipeline status, failures, and execution duration. Compare source and destination record counts, validate required fields, and review rejected records to ensure data completeness.